

Design Review

Submission Detail	Student Entry
GROUP ID	
GITHUB REPO LINK	

Purpose

Document the system requirements, hardware connections, RTOS task structure, behavioural models, assumptions, and AI-supported design reasoning before major implementation. Treat this Week 6 submission as an architecture freeze before full integration and Tribunal 1. The design may change later, but major changes must be justified with engineering evidence.

How to use this template

- This is an architecture-freeze document. The team may improve the design later, but major changes must be justified during Tribunal 1.
- Use AI assistants for ideation, critique, and gap checking. Do not paste AI output directly as the final design.
- Summarise, challenge, correct, and justify AI suggestions using engineering reasoning.
- Keep all artefacts consistent: requirements, pin layout, RTOS task table, task interaction diagram, statecharts, assumptions, and AI evidence.

Team Role and Subsystem Ownership

Identify each buddy's primary subsystem responsibility. The role does not need to match the examples exactly, but every major hardware and software responsibility should have a clear owner.

Buddy	Student ID	Primary subsystem / responsibility
Buddy 1		Example: communications, telemetry, or remote interface
Buddy 2		Example: motion, actuator control, or timing-critical outputs
Buddy 3		Example: sensors, line tracking, barcode, or perception inputs
Buddy 4		Example: safety monitor, fault detection, or environmental sensing
Buddy 5		Example: integration, obstacle handling, or system coordination

Submission Instructions

- Complete every required section.
- Diagrams may be produced using any suitable tool, but labels must remain readable when inserted into this document.
- AI assistants may support drafting. The team remains responsible for technical correctness, consistency, and the ability to explain every decision.

- Use consistent names across the requirements, hardware sketch, task table, task interaction diagram, and statecharts.

1. Requirements

State what the system must do and the important quality or timing constraints. Requirements should be specific, testable, and traceable to the proposed design (you may add more FR and/or NFR as needed).

1.1 Functional Requirements

ID	Example Requirement
FR1	The oven shall allow the user to select a cooking mode.
FR2	The oven shall regulate the internal temperature automatically.
FR3	The oven shall monitor the cooking duration.
FR4	The oven shall notify the user when cooking is complete.
FR5	The oven shall detect unsafe operating conditions and disable heating.

1.2 Non-Functional Requirements

ID	Example Requirement
NFR1	The temperature shall be sampled every 100 ms during operation.
NFR2	The user interface shall respond to a button press within 200 ms.
NFR3	A critical over-temperature event shall cause the heating element to be disabled.

2. Hardware Integration Pin Layout

Sketch how sensors, actuators, displays, communication modules, and power connections interface with the Robo Pico. Label pins and interfaces such as GPIO, ADC, PWM, I2C, SPI, and UART.

Device / Module	Type	Signal Name	Sensor label	Pico GPIO Pin	Interface	Notes / RTOS Task Owner
Oven temperature sensor	Sensor	TEMP_ADC	OUT / AO	GP26	ADC	Sampled by Temperature Sensor Task every 100 ms.
Door switch	Sensor	DOOR_SW	DO / SIG	GP2	GPIO input	Monitored by Safety Monitor Task; open door disables heating.
Start / stop button	User input	START_STOP	SW / SIG	GP3	GPIO input	Read by User Interface Task; debounce in software.
OLED display	Display	SDA	SDA	GP4	I2C	Updated by User Interface Task; shared bus requires ownership rule.
OLED display	Display	SCL	SCL	GP5	I2C	Clock line for the shared I2C display bus.
Heater relay / MOSFET	Actuator	HEATER_EN	IN / EN	GP10	GPIO output	Controlled only by Heater Driver Task; forced off by safety event.
Cooling fan	Actuator	FAN_PWM	PWM / IN	GP11	PWM	Driven by Cooking Control or Heater Driver Task based on temperature state.
Buzzer	Actuator	BUZZER_PWM	SIG / PWM	GP12	PWM / GPIO	Owned by Notification Task for completion and fault alerts.

3. RTOS Task Table

List the ALL the proposed RTOS tasks. Priorities are preliminary at Week 6 and must be justified.

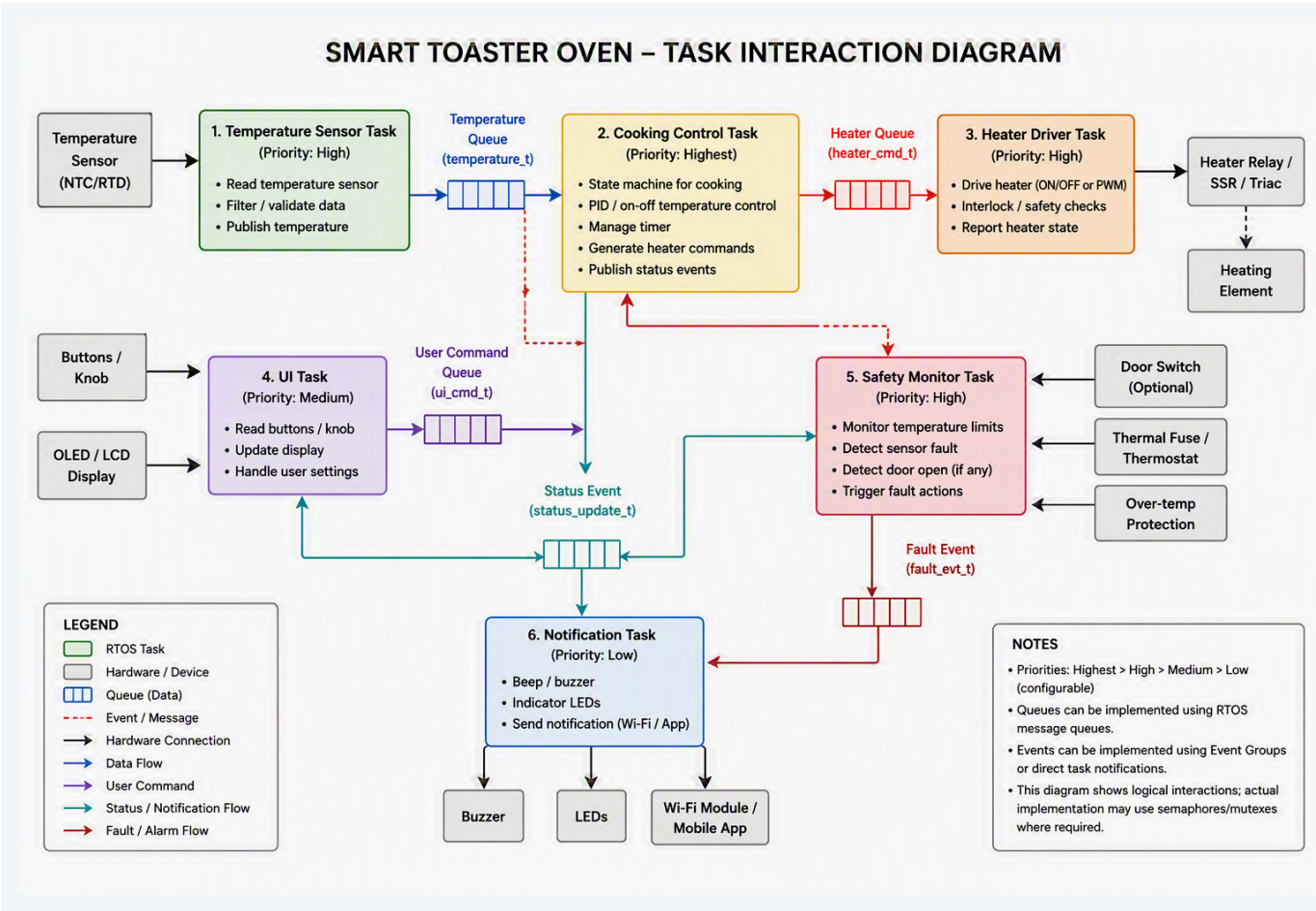
Task	Purpose	Inputs	Outputs	Trigger	Priority
Temperature Sensor	Sample oven temperature	Temperature sensor	Temperature queue	Periodic, 100 ms	High
Cooking Control	Determine heating action	Temperature and user settings	Heater commands	Event-driven	High
Heater Driver	Control heating element	Heater commands	Relay or PWM	Event-driven	High
User Interface	Handle controls and display	Buttons / encoder, status	User settings, display	Event / periodic	Medium
Safety Monitor	Detect unsafe conditions	Temperature and fault signals	Shutdown / alarm event	Periodic / event	Highest
Notification	Notify the user	Completion or fault event	Buzzer / app message	Event-driven	Low

Priority and ownership justification:

Enter justification here

4. Task Interaction Diagram

Show RTOS tasks, queues, semaphores, mutexes, event groups, notifications, and data flow. Clearly distinguish software tasks from hardware devices and interrupt service routines.

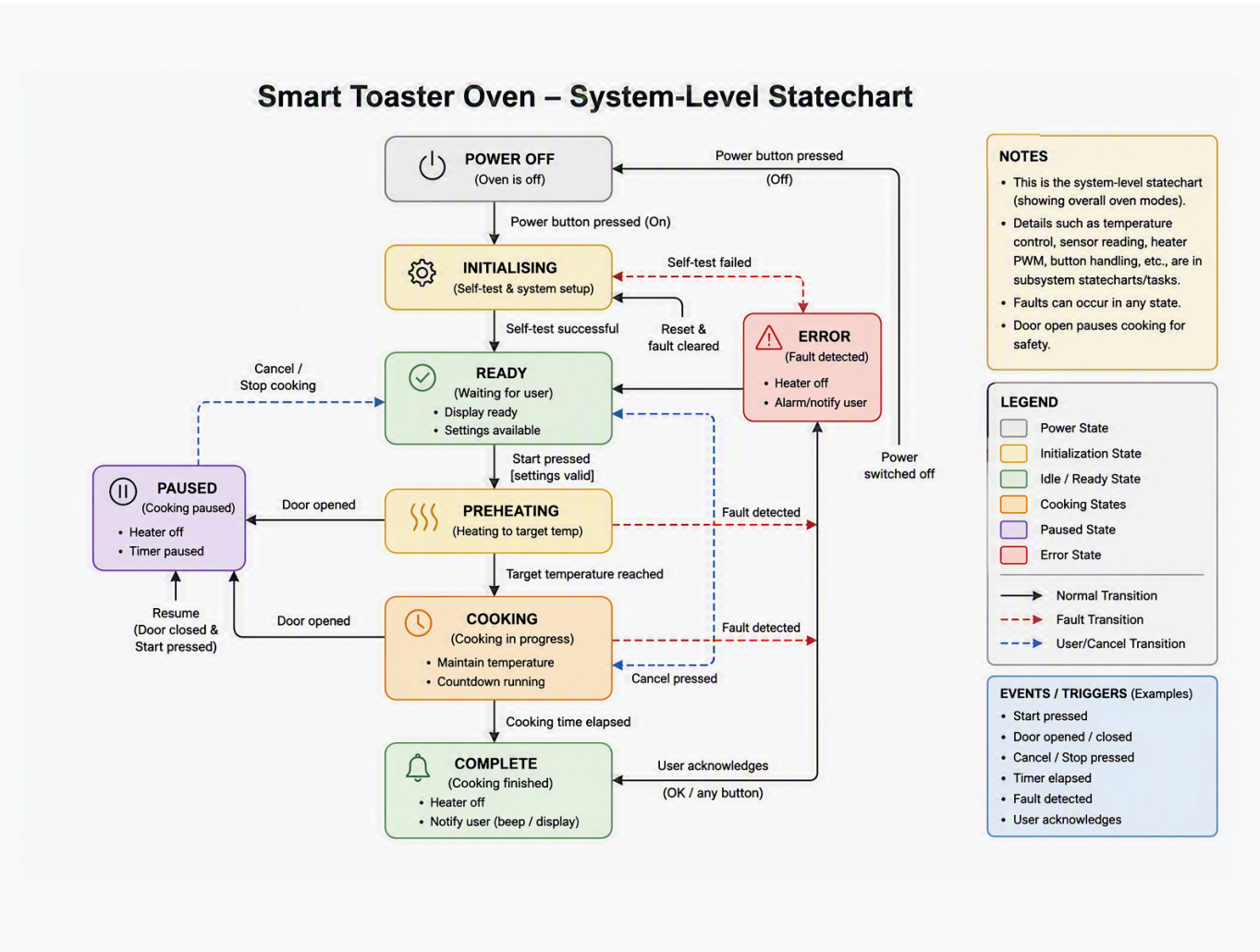


Brief explanation of the main communication paths:

Enter explanation here

5. System-Level Statechart

Model the operating states of the entire system. Label transition events or conditions. Include relevant fault, stop, or recovery behaviour.



Explain the main states and any important assumptions:

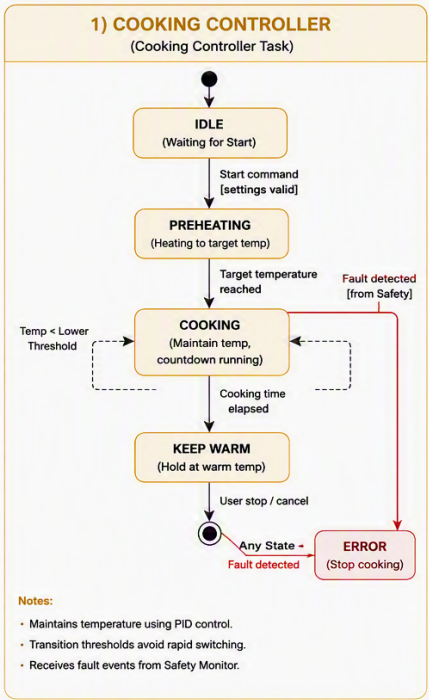
Enter explanation here

6. Subsystem Statecharts

Add more sub-systems if needed.

6.1 Subsystem Statechart 1

Field	Student Entry
Subsystem name	
Related RTOS task(s)	
Purpose	
Reason a statechart is needed	

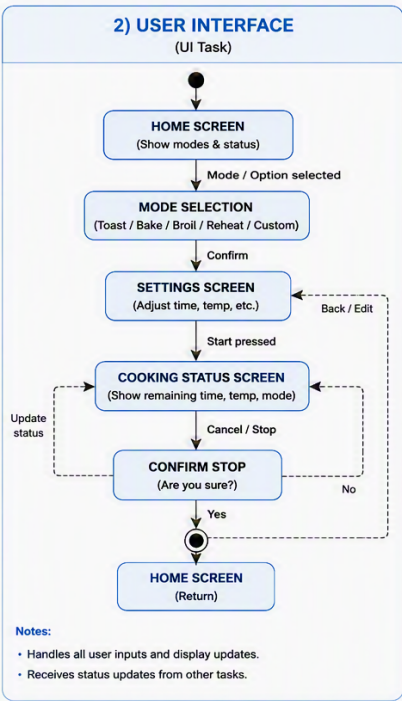


Brief explanation of states, events, and fault handling:

Enter explanation here

6.2 Subsystem Statechart 2

Field	Student Entry
Subsystem name	
Related RTOS task(s)	
Purpose	
Reason a statechart is needed	



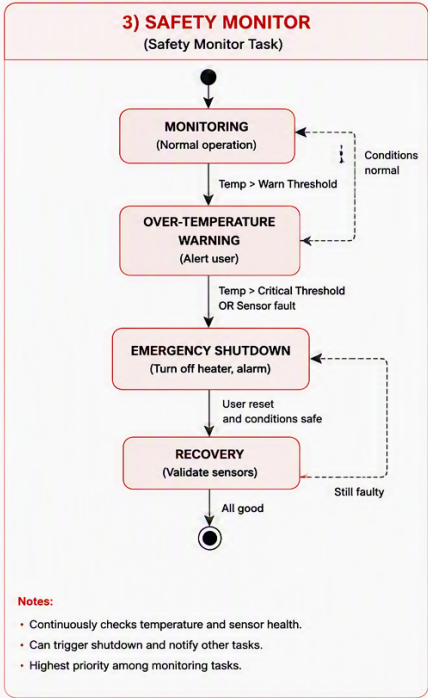
Brief explanation of states, events, and fault handling:

Enter explanation here

6.3 Subsystem Statechart 3

Field	Student Entry
Subsystem name	
Related RTOS task(s)	

Field	Student Entry
Purpose	
Reason a statechart is needed	



Brief explanation of states, events, and fault handling:

Enter explanation here

7. Assumptions

List assumptions that the design depends on. Each assumption must state the risk if it is wrong and how the team will validate it before full integration. Have **at least** 1 assumption per buddy.

ID	Assumption	Risk if wrong	Validation method	Owner	Status
A1	The oven temperature sensor gives stable readings when sampled every 100 ms.	Cooking control may react to noisy or stale temperature values, causing poor heating regulation.	Log temperature readings for several minutes at room temperature and during a controlled heating test; check for excessive noise or missed samples.	Buddy 1	Open
A2	The door switch reliably detects open and closed states without frequent false transitions.	The heater may be disabled unnecessarily, or worse, remain enabled when the door is open.	Run repeated open-close tests and verify that the Safety Monitor Task disables heating immediately when the door is opened.	Buddy 2	Open
A3	The user interface can process button presses and update the display within 200 ms.	The oven may feel unresponsive, and users may press buttons repeatedly or select the wrong mode.	Measure response time from button press to display update using timestamps, serial logs, or video evidence.	Buddy 3	Open
A4	The heater relay or MOSFET switches safely and only responds to commands from the Heater Driver Task.	Conflicting commands may turn the heater on during a fault condition or fail to turn it off when required.	Test normal start, stop, temperature-limit, and fault-shutdown scenarios; confirm that safety events override cooking commands.	Buddy 4	Open
A5	The OLED display and any other I2C devices can share the bus without blocking time-critical safety or temperature tasks.	Long display updates or bus contention may delay temperature sampling or safety checks.	Stress-test frequent display updates while logging Temperature Sensor and Safety Monitor task timing.	Buddy 5	Open

8. AI-Supported Design Evidence

Document how AI was used to support the design. The aim is not to show that AI produced perfect answers, but to show that the team used AI critically, corrected weak suggestions, and made final engineering decisions.

8.1 Pre-Submission AI Ideation

Buddy	AI prompt or discussion used	Useful AI ideas	Weak or unrealistic AI ideas	Team decision
Buddy 1	Asked AI to suggest possible RTOS tasks and data flow for a smart toaster oven with temperature sensing, heating control, user input, display, buzzer, and safety monitoring.	AI suggested separating temperature sensing, cooking control, heater driving, user interface, safety monitoring, and notification into different tasks.	AI initially suggested adding Wi-Fi notifications and recipe database features, which were too ambitious for the Week 6 prototype.	The team kept the core RTOS task structure and deferred optional connectivity features until the basic toaster behaviour is working reliably.
Buddy 2	Asked AI what safety conditions a toaster oven should detect and how these should affect the RTOS task priorities.	AI identified door-open detection, over-temperature detection, sensor failure, and forced heater shutdown as important safety behaviours.	AI assumed a real mains-powered heater could be directly controlled, which is unsafe and not suitable for the laboratory prototype without proper isolation.	The team decided that the Safety Monitor Task should have the highest priority and that all heating outputs must be demonstrated using a safe low-voltage relay, LED, or simulated heater.
Buddy 3	Asked AI to propose a simple user interface state flow for selecting cooking mode, setting duration, starting cooking, stopping cooking, and showing completion.	AI suggested useful states such as Idle, Mode Select, Cooking, Paused, Complete, and Fault, which helped shape the system-level statechart.	AI suggested too many menu levels and settings, which would make the prototype harder to implement and explain by Week 6.	The team kept a simple interface with start/stop, mode selection, duration setting, status display, and completion alert.
Buddy 4	Asked AI to identify possible shared resources and timing risks in the toaster RTOS design.	AI highlighted possible contention on the I2C display bus and the need to avoid display updates delaying safety or temperature sampling.	AI suggested using many mutexes and queues without explaining which shared resources actually required protection.	The team decided to use queues for sensor and command data, and to restrict direct display access to the User Interface Task.
Buddy 5	Asked AI to review whether the requirements, hardware pin layout, RTOS task table, task interaction diagram, and statecharts were consistent with one another.	AI helped identify traceability links, such as mapping temperature regulation to the Temperature Sensor, Cooking Control, and Heater Driver tasks.	AI sometimes invented extra sensors and mobile app features that were not in the team's selected hardware list.	The team accepted only suggestions that matched the selected hardware and updated the design artefacts to use consistent names for tasks, signals, and states.

8.2 Team-Reviewed Architecture Freeze

AI or initial idea	Team decision	Engineering reason	Affected artefact
Separate the toaster oven software into Temperature Sensor, Cooking Control, Heater Driver, User Interface, Safety Monitor, and Notification tasks.	Accepted with minor simplification	The split gives each time-critical function a clear responsibility while keeping the design small enough for a Week 6 prototype. It also supports easier testing of sensing, control, safety, and user feedback.	RTOS task table, task interaction diagram, subsystem statecharts
Give the Safety Monitor Task the highest priority and allow it to force the heater output off during door-open, over-temperature, or sensor-failure conditions.	Accepted	Safety behaviour must override normal cooking behaviour. A lower-priority safety task could allow unsafe heating to continue while the system is busy with control, display, or notification work.	Functional requirements, non-functional requirements, task priority table, Safety Monitor statechart, assumptions
Use Wi-Fi notifications, recipe storage, and mobile-app control as part of the toaster oven design.	Deferred	These features increase complexity but are not necessary to demonstrate RTOS task design, timing, safety handling, or state-based control by Week 6.	Requirements, task table, AI ideation notes, design scope
Allow multiple tasks to directly access the OLED display and I2C bus.	Modified	Direct access from many tasks can create timing delays and unclear ownership. The team restricted display updates to the User Interface Task and used queues to pass status information.	Hardware pin layout, task interaction diagram, assumptions, design consistency check

8.3 Final AI-Vetted Gap Check

After completing the design, ask AI to act as a strict embedded systems reviewer. The review should identify missing requirements, unclear interfaces, untested assumptions, weak mitigations, timing concerns, task conflicts, or resource contention.

Use one of these Team Response options: **Fixed, Partially fixed, Accepted as risk, Deferred, Rejected with reason, or Needs validation.** The action and evidence columns must justify the selected response.

AI reviewer prompt used

Act as a strict embedded systems reviewer for a smart toaster oven design. Check the requirements, pin layout, RTOS task table, task interaction diagram, statecharts, assumptions, and AI evidence. Identify missing requirements, unclear interfaces, untested assumptions, weak safety mitigations, timing risks, task conflicts, and shared-resource problems. Do not rewrite the design; list only gaps that the team must review and justify.

AI-identified gap	Team response	Action taken	Evidence or reason
The design states that unsafe conditions disable heating, but it does not clearly define all unsafe	Fixed	Added door-open, over-temperature, and temperature-sensor failure as explicit fault cases. Updated the Safety Monitor statechart to send the system to	FR5, NFR3, the Safety Monitor Task, and Assumption A2/A4 now describe how faults disable

conditions and recovery behaviour.		Fault until the condition is cleared and the user resets the oven.	heating and how this will be validated.
The display and user interface may share the I2C bus or CPU time with more critical tasks, creating possible timing delays.	Needs validation	Assigned display updates only to the User Interface Task and used queues for passing status information. The team still needs to stress-test display updates while logging safety and temperature task timing.	The hardware pin layout notes that the OLED uses I2C, while Assumption A5 records the timing risk and planned validation method.
The heater output could be interpreted as controlling a real mains-powered heating element, which is unsafe for a student prototype.	Accepted as risk	Kept the heater output in the design but explicitly limited it to a safe low-voltage relay, LED, or simulated heater output. The team will not directly switch mains power in the Week 6 prototype.	The risk is acceptable only because the Week 6 prototype uses a simulated or low-voltage output. Assumption A4 requires validation before any actuator integration.
The relationship between requirements, tasks, and statecharts may not be obvious unless the team explains traceability across artefacts.	Partially fixed	Mapped FR1 to User Interface, FR2 to Temperature Sensor/Cooking Control/Heater Driver, FR3 to Cooking Control, FR4 to Notification, and FR5 to Safety Monitor. A more complete traceability table can be added after Week 6 if implementation changes.	The current mapping is sufficient for the architecture-freeze review, but the team should update traceability after task periods, queues, and state transitions are confirmed.

9. Design Consistency

Use this section to demonstrate that the design artefacts describe one coherent system rather than independent diagrams.

Check	Student Response
Which task owns each hardware device?	Temperature Sensor Task owns the oven temperature sensor. Safety Monitor Task owns the door switch and fault checks. User Interface Task owns the start/stop button and OLED display. Heater Driver Task owns the heater relay or simulated heater output. Cooking Control or Heater Driver handles the cooling fan based on the temperature state. Notification Task owns the buzzer.
Which task has the highest priority, and why?	Safety Monitor has the highest priority because it must detect unsafe conditions and force the heater off before normal cooking or display updates continue. Temperature Sensor, Cooking Control, and Heater Driver are also high priority because they affect heating behaviour and timing. User Interface is medium priority, while Notification is low priority.
Which subsystem statechart corresponds to which RTOS task or subsystem?	The Cooking Controller statechart corresponds to the Cooking Control and Heater Driver tasks. The User Interface statechart corresponds to the User Interface Task and Notification Task. The Safety Monitor statechart corresponds to the Safety Monitor Task and covers door-open, over-temperature, sensor-failure, forced shutdown, and reset behaviour.
How are functional requirements reflected in the task and statechart design?	FR1 is handled by the User Interface Task and Mode Select state. FR2 is handled by the Temperature Sensor, Cooking Control, and Heater Driver tasks. FR3 is handled by the Cooking Control Task and Cooking state. FR4 is handled by the Notification Task and Complete state. FR5 is handled by the Safety Monitor Task and Fault state.
What changes are still expected after the Week 6 review?	After Week 6, the team expects to refine task periods, confirm priority values using timing logs, validate assumptions A1 to A5, simplify any statechart transitions that are difficult to implement, and update the task interaction diagram if queues, event groups, or shared-resource rules change during integration.